

PAROLE CHIAVE:

ROUTINE → BLOCCO DI CODICE CHE SVOLGE UN COMPITO.

FUNZIONE → È UNA ROUTINE CON PARAMETRI E RESTITUISCONO UN VALORE.

PROCEDURA $\rightarrow$ È UNA ROUTINE CHE ACCETTA PARAMETRI MA NON RESTITUISCE NULLA.

CALL SITE → POSIZIONE IN CUI SI CHIAMA LA ROUTINE.

CALLER ROUTINE → LA ROUTINE CHE INVoca UN'ALTRA ROUTINE.

CALLE ROUTINE → LA ROUTINE CHE VIENE CHIAMATA DALL'ALTRA ROUTINE.

UNA ROUTINE È DEFINITA DA LABEL $\Rightarrow$. LABEL

LE INVOCAZIONI DI FUNZIONE VENGONO

FATTE ATTRAVERSO JAL, PERCHÉ

.END

SALVA $PC+4$ IN UN REGISTRO E EFFETUA IL SALTO ALLA FUNZIONE (INIZIO INDICATO DALLA LABEL) INFINE IL REGISTRO CONTENENTE $PC+4$ VERRÀ USATO PER TORNARE ALLA **CALLER ROUTINE**.

ABI (APPLICATION BINARY INTERFACE) → DEFINISCE UNA SERIE DI CONVENZIONI, CHE DEFINISCE COME LE ROUTINE DEVONO INTERAGIRE CON L'AMBIENTE DI ESECUZIONE, STABILENDO REGOLE SU COME:

- VENGONO PASSATI I PARAMETRI.
- VENGONO RESTITUITI I VALORI.
- VENGONO GESTITI I REGISTRI.
- SI EFFETTUA IL SALVATAGGIO DEI DATI IN MEMORIA.
- È DEFINITO IL MEMORY LAYOUT.

COME VENGONO PASSATI I PARAMETRI AD UNA FUNZIONE?

PER CONVENZIONE, I PRIMI 8 PARAMETRI, VENGONO PASSATI, GRAZIE AI REGISTRI AO...A7, SE SONO PIÙ DI 8 VENGONO INSERITI NELLO STACK.

ES: $\text{INT SUM}_{10}(\overbrace{\text{INT A, INT B, \dots, INT J}}^{10 \text{ VALORI}})$

COME SI PROCEDE?

MAIN :

LI A0, 10
LI A1, 20
LI A2, 30
LI A3, 40
LI A4, 50
LI A5, 60
LI A6, 70
LI A7, 80

} I PRIMI 8 PARAMETRI

ADDI SP, SP, -8

ALLOCO LO SPAZIO PER 2 PARAMETRI (4 BYTE PER OGNUNO)
LO STACK CRESCE VERSO GLI INDIRIZZI + BASSI

LI T1, 100

SW T1, 4(SP)

LI T2, 90

SW T2, 0(SP)

} METTO GLI ULTIMI 2 PARAMETRI NELLO STACK MA IN ORDINE INVERSO, PERCHÉ QUANDO FARO POPC(), AVRÒ GLI ELEMENTI DAL 9 IN POI, IN ORDINE.

JAL SUH10

INVOCHIAMO LA FUNZIONE

ADDI SP, SP, 8

DEADUO CHIAMO I PARAMETRI DALLO STACK, SPETTA PROPRIO AL CALLER QUESTO COMPITO

RET

SUH10 :

LW T1, 0(SP)

PRENDU IL 9° PARAMETRO

LW T2, 4(SP)

IL 10° PARAMETRO

ADD A0, A0, A1

ADD A0, A0, A2

ADD A0, A0, A3

SI UTILIZZA A0 COME VALORE DI RITORNO

ADD A0, A0, A4

IN CASO IL VALORE DI RITORNO SIA DI 64 BIT

ADD A0, A0, A5

VIENE USATO A1 PER CONTENERE I 32 BIT +

ADD A0, A0, A6

SIGNIFICATIVI.

ADD A0, A0, A7

IN CASO A0 O A1 FOSSERO SERVITI AL MAIN,

ADD A0, A0, T1

SAREBBE STATO IL SUO COMPITO QUELLO DI SALVAR

ADD A0, A0, T2

LI NELLO STACK

RET

PASSAGGIO PER VALORE E PER RIFERIMENTO:

```
INT INC (INT V) {  
    RETURN V+1;  
}  
=>  
INC:  
    ADDI A0, A0, 1    # V = V+1  
    RET
```

```
VOID INC (INT *V) {  
    *V = *V+1;  
}  
=>  
INC:  
    LW A1, 0(A0)    # x = *V  
    ADDI A1, A1, 1    # x = x+1  
    SW A1, 0(A0)    # *V = x  
    RET
```

IN RISC-V ABBIAMO VISTO ANCHE LE VARIABILI GLOBALI:

```
INT X;  
INT MAIN () {  
    RETURN X+1;  
}  
=>  
.DATA  
X: .WORD 0  
.TEXT  
MAIN:  
    LA A0, X  
    LW A0, 0(A0)  
    ADDI A0, A0, 1  
    RET
```

LE VARIABILI LOCALI

```
INT FOO () {  
    INT TEMP = 5;  
    RETURN TEMP+1;  
}  
=>  
FOO:  SI PREFERISCE USARE I  
    REGISTRI  
    LI A0, 5    # TEMP = 5  
    ADDI A0, A0, 1  
    RET
```

NON SI POSSONO USARE SEMPRE I REGISTRI QUANDO:

- UNA ROUTINE HA UN ELEVATO NUMERO DI VARIABILI LOCALI.
- LA VARIABILE LOCALE È UN ARRAY O UNA STRUTTURA.
- IL CODICE HA BISOGNO DELL'INDIRIZZO DELLA VARIABILE LOCALE.

(PASSAGGIO PER RIFERIMENTO) → PERCHÉ SE SALVO UN DATO IN UN REGISTRO, QUESTO NON HA UN INDIRIZZO, INVECE NELLO STACK SÌ.

```

INT Foo() {
    INT ARRAY[4] = {1, 2, 3, 4};
    RETURN ARR[2];
}

```

INSERISCO TUTTO
IL MIO ARRAY
NELLO STACK, ALMENO
HANNO ANCHE UN
INDIRIZZO

Foo:

```

ADDI SP, SP, -16 # 4 ELEMENTI
LI T0, 1
SW T0, 0(SP) # ARRAY[0] = 1
LI T0, 2
SW T0, 4(SP) # ARRAY[1] = 2
LI T0, 3
SW T0, 8(SP)
MV A0, T0 # METTO SUBITO A0 = ARRAY[2]
LI T0, 4 → SALVO IL VALORE
SW T0, 12(SP)
ADDI SP, SP, 16 # DEALLOCO
RET → RITORNO INDIETRO

```

```

STRUCT Point {
    INT x, y;
}

```

```

INT Foo {
    STRUCT POINT P = {3, 4};
    RETURN P.x + P.y;
}

```

CARICO NELLO STACK

Foo:

```

ADDI SP, SP, -8 # ALLOCO PER 2 VALORI
LI T0, 3
LI T1, 4
SW T0, 0(SP) # P.x = 3
SW T1, 4(SP) # P.y = 4
LW T0, 0(SP) # T1 = P.x
LW T1, 4(SP) # T2 = P.y
ADD A0, T1, T2 # A0 = T1 + T2
ADDI SP, SP, 8 # DEALLOCO
RET

```

IN QUESTI CASI SEMBRA SENZA
SENSO, HA HETTEMPO CASO CHE
LE STRUTTURE SIANO MOLTO PIÙ
GRANDI, OPPURE CHE I REGISTRI
SONO FINITI LO STACK CI PUÒ AIUTARE.

ALCUNI REGISTRI DEVONO ESSERE PRESERVATI DAL CALLER ALTRI DAL CALLEE.

AD ESEMPIO I REGISTRI: A0 ... A7 SONO CALLER SAVED.

METTIAMO CASO:

SUM:

ADD A0, A0, A1 # $x = x + y$

ADD A0, A0, A2 # $x = x + z$

RET

SE A0, A1, A2 FOSSERO STATI
IMPORTANTI PER IL CALLER DOVEVA
ESSERE LO STESSO A SALVARLI
NELLO STACK.

MAIN:

...

ADDI SP, SP, -12 # ALLOCO

SW A0, 0(SP)

SW A1, 4(SP)

SW A2, 8(SP)

} SALVO NELLO STACK

JAL SUM # RICHIAMO LA FUNZIONE

LW A0, 0(SP)

LW A1, 4(SP)

LW A2, 8(SP)

} SE LI RIPRENDE

ADDI SP, SP, 12

STESSA COSA VALE PER I REGISTRI TEMPORANEI OVERO T0-T6

ANCHE IL REGISTRO RA CHE CONTIENE L'INDIRIZZO DI RITORNO DEVE
ESSERE SALVATO DAL CALLER, IN MODO TALE DA PERMETTERE DI
RITORNARE INDIETRO ALL CALL SITE.

METTIAMO CASO:

```
INT SUM1 (INT x, INT y) {
```

```
    RETURN x + y;
```

```
}
```

```
INT SUM2 (INT A, INT B, INT C) {
```

```
    INT RESULT = SUM1(A, B);
```

```
    RETURN RESULT + C;
```

```
}
```

```
INT MAIN() {
```

```
    INT x = 3, y = 4, z = 5;
```

```
    INT RESULT = SUM2(x, y, z);
```

```
    PRINTF("%d\n", RESULT);
```

```
    RETURN 0;
```

```
}
```

MAIN :

```
LI A0, 3  
LI A1, 4  
LI A2, 5
```

} PASSO 1 3 PARAMETRI

JAL SUM2

SUM2 :

ADDI SP, SP, -4 # ALLOCO SPAZIO

SW RA, 0(SP) # SALVO **RA** NELLO STACK

```
MV A0, A0  
MV A1, A1
```

} # x
y
INUTILE MA DA FARE QUANDO I VALORI SONO DIVERSI

JAL SUM1 # MODIFICA **RA**

LW RA, 0(SP) # RIPRENDO IL **RA** ORIGINALE

ADDI SP, SP, 4 # DEALLOCO

ADD A0, A0, A2 # RESULT + Z

RET # USA IL **RA** ORIGINALE

SUM1:

ADD A0, A0, A1 # RESULT = x+y

RET # USA IL NUOVO **RA**

| REGISTRI **SO-S11** SONO REGISTRI CHE DEVONO ESSERE SALVATI DAL **CALLER**, È SUA RESPONSABILITÀ SALVARLI.

INOLTRE SONO REGISTRI **SALVABILI**, NON VENGONO DISTRUTTI AD OGNI CHIAMATA. METTIAMO CASO IL MAIN UTILIZZI I REGISTRI **SO-S11** :

MAIN:

...

JAL COMPLEX-ADD:

COMPLEX-ADD

ADDI SP, SP, -8 # ALLOCO

SW RA, 0(SP) # SALVO **RA** **OBBLIGATORIAMENTE**

SW S0, 4(SP) # SALVO **S0** PERCHÉ VOGLIO USARLO (MA NON DEVE INIACIARE IL **RA**)

ADD S0, A0, A1 # SALVO IN S0 = A+B

LI A0, 5 # PREPARO L'ARGOMENTO

JAL DUMMY-FUNCTION # CHIAMO UNA FUNZIONE CASUALE

ADD A0, A0, S0 # SALVO IN A0 = S0 + RES_DUMMY-FUNCTION

LW RA, 0(SP) }
LW S0, 4(SP) } MI RIPRENDO RA E S0

ADDI SP, SP, 8 # DEALLOCO

RET

I REGISTRI GP/TP SONO FONDAMENTALI PER THREAD E ACCESSO ALLE
VARIABILI GLOBALI, L'ABI STANDARD IMPONE CHE NON VENGANO MODIFICATI
DALLE ROUTINE.